Introduction to Databases & SQL

Learn how data powers everything from social media to banking systems. This course will help you understand how information is stored, structured, and retrieved using databases and SQL.

What is a Database?

A database is a structured collection of data organized for efficient access, management, and updates. Think of it as a digital filing system that keeps information neatly organized and easy to find.

Table

The central structure in a relational database, similar to an Excel spreadsheet with rows and columns

Row (Record)

A single entry in a table representing one specific entity, like a particular student or customer

Column (Field)

A specific attribute of the data, such as Email Address, Date of Birth, or Phone Number

Core Concepts: Keys & Relationships

Primary Key (PK)

A unique identifier for every record in a table. It cannot be NULL and ensures no duplicate rows exist.

- StudentID in a students table
- SocialSecurityNumber for employee records
- ProductID in inventory systems

Table Relationships

How tables connect and reference each other to create meaningful data structures.

- **One-to-One:** One student has one transcript
- **One-to-Many:** One teacher has many students
- **Many-to-Many:** Students enroll in many classes; classes have many students

SQL vs. NoSQL

Two fundamentally different approaches to storing and retrieving data. Understanding when to use each type is crucial for building efficient applications.

SQL (Relational)

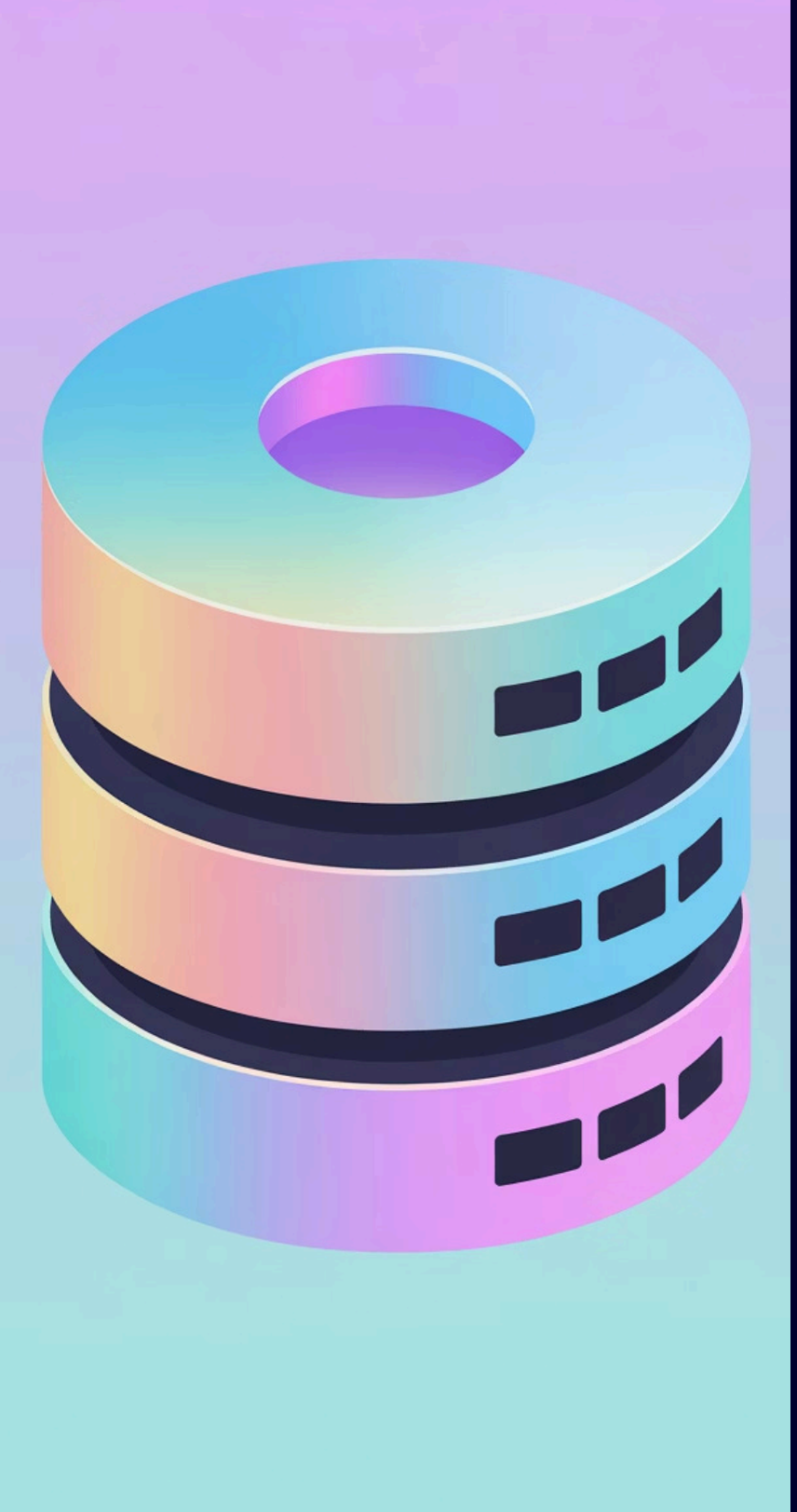
Structured Query Language with schema-based, strict structure and table-oriented design

- SQL Server, PostgreSQL, MySQL
- Best for: Financial systems, inventory, complex relationships
- ACID compliant transactions

NoSQL (Non-Relational)

Flexible schema with dynamic structure, document or key-value oriented design

- MongoDB, Cassandra, Redis
- Best for: Real-time big data, content management, rapid prototyping
- Horizontal scaling



Setting Up SQL Server Environment

SQL Server Engine

The database engine serves as the "brain" that processes queries and manages data storage

SSMS Interface

SQL Server Management Studio is your interface tool for writing code and viewing data

Installation

Download SQL Server Developer Edition (free for learning) and install SSMS from Microsoft

Connect

Connect to localhost or . (dot) using Windows Authentication to start working

SQL Data Types

Choosing the right data type ensures efficient storage and prevents data corruption. Here are the most common types you'll use:

INT

Whole numbers like 1, 100, or 2020. Perfect for IDs, counts, and numeric values without decimals

VARCHAR(n)

Variable-length text where n is the maximum characters. Use VARCHAR(50) for names or VARCHAR(255) for descriptions

DATETIME

Stores date and time values like 2026-12-31 14:30:00. Essential for tracking when records were created or modified

FLOAT(n)

Approximate numbers with scientific notation. Great for scientific data where precision is less critical than range

BIT

Boolean values stored as 0 or 1, representing False or True. Ideal for yes/no flags and binary states

Creating a Database & Table

Before storing data, you need to create the container (database) and define its structure (table). This establishes the blueprint for how information will be organized.

CREATE DATABASE

Sets up a new container to hold all your related data tables

CREATE TABLE

Defines the structure by specifying columns, data types, and constraints

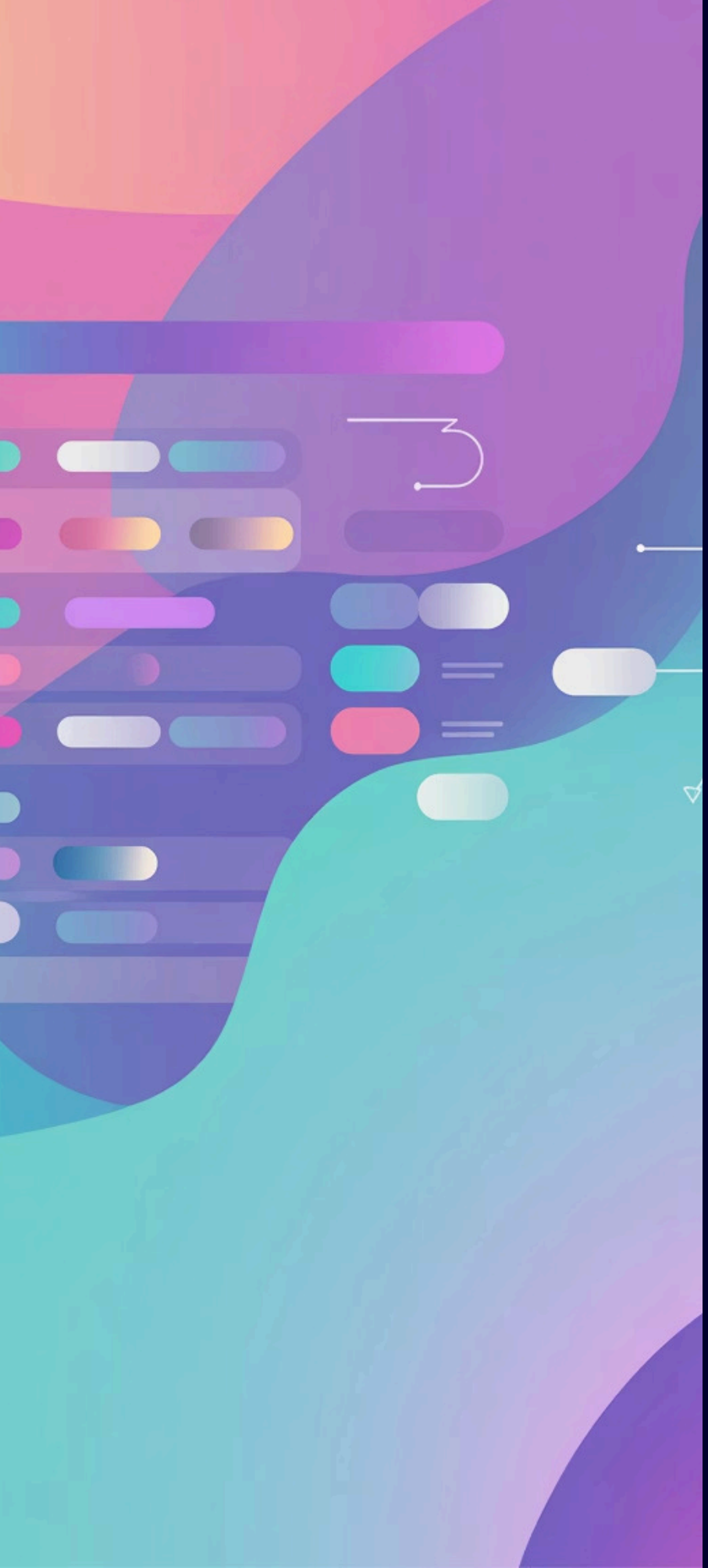
Example

```
-- Create the container
CREATE DATABASE UniversityDB;

-- Create the structure
CREATE TABLE Students (
 StudentID INT PRIMARY KEY,
 FirstName VARCHAR(50),
 LastName VARCHAR(50),
 EnrollmentDate DATETIME
);
```

Populate Table

```
INSERT INTO Students (StudentID, FirstName, LastName, EnrollmentDate) VALUES (1, 'Alice', 'Johnson', '2023-09-01 09:00:00'), (2, 'John', 'Smith', '2023-09-01 10:30:00'), (3, 'Charlie', 'Davis', '2023-09-02 08:45:00'), (4, 'Diana', 'Prince', '2023-09-02 11:15:00'), (5, 'Ethan', 'Hunt', '2023-09-03 14:00:00'), (6, 'Fiona', 'Gallagher', '2023-09-03 15:30:00'), (7, 'George', 'Miller', '2023-09-04 09:20:00'), (8, 'Hannah', 'Abbott', '2023-09-04 13:45:00'), (9, 'Ian', 'Wright', '2023-09-05 10:00:00'), (10, 'Julia', 'Roberts', '2023-09-05 16:20:00');
```



Basic SELECT Queries

The SELECT statement is your primary tool for retrieving data. It lets you specify exactly which information you want to see from your tables.

SELECT

Choose which columns you want to retrieve from the table

FROM

Specify the table source where the data resides

Asterisk (*)

Selects all columns—useful for quick checks during development

Query Examples

```
-- Get all data from Students table
SELECT * FROM Students;

-- Retrieve specific columns only
SELECT FirstName, LastName
FROM Students;
```

Filtering & Sorting Results

Raw data isn't always useful. Use filtering and sorting to focus on relevant information and present it in a meaningful order.

1

WHERE Clause

Filters rows based on a specified condition. Think of it as a search filter that narrows down results to only matching records.

2

ORDER BY Clause

Sorts the results in ascending (ASC) or descending (DESC) order. Organize data chronologically, alphabetically, or by numeric value.

3

LIMIT / TOP

Restricts the number of rows returned. SQL Server uses TOP; other databases use LIMIT for the same purpose.

Example Queries

```
-- Find students named 'John'
SELECT * FROM Students
WHERE FirstName = 'John';

-- Show 5 most recently enrolled
SELECT TOP 5 * FROM Students
ORDER BY EnrollmentDate DESC;
```

Hands-on Activity

Practice what you've learned by building a Courses table and writing queries to retrieve specific information. This exercise will reinforce your understanding of database creation and SQL commands.

01

Create Courses Table

Build a new table named Courses with columns: CourseID (INT), CourseName (VARCHAR), and Credits (INT)

02

Insert Sample Data

Add dummy course data including course names and credit values. We'll learn INSERT syntax together

03

Query Filtered Results

Write a SELECT query to fetch only courses worth more than 3 credits using a WHERE clause

04

Sort Alphabetically

Write another query to list all courses ordered by name from A to Z using ORDER BY

Answer Script

Here are the SQL code examples for the hands-on activity, demonstrating database creation, data insertion, filtering, and sorting techniques.

Task 1: Create Courses Table

```
CREATE TABLE Courses (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(100),
    Credits INT
);
```

Task 2: Insert Sample Data

```
INSERT INTO Courses (CourseID, CourseName, Credits) VALUES
    (1, 'Introduction to Computer Science', 4),
    (2, 'Calculus I', 4),
    (3, 'English Composition', 3),
    (4, 'Art History', 2),
    (5, 'Database Systems', 3);
```

Task 3: Query Filtered Results (Credits > 3)

```
SELECT *
    FROM Courses
    WHERE Credits > 3;
```

Task 4: Sort Alphabetically (ORDER BY CourseName ASC)

```
SELECT *
    FROM Courses
    ORDER BY CourseName ASC;
```